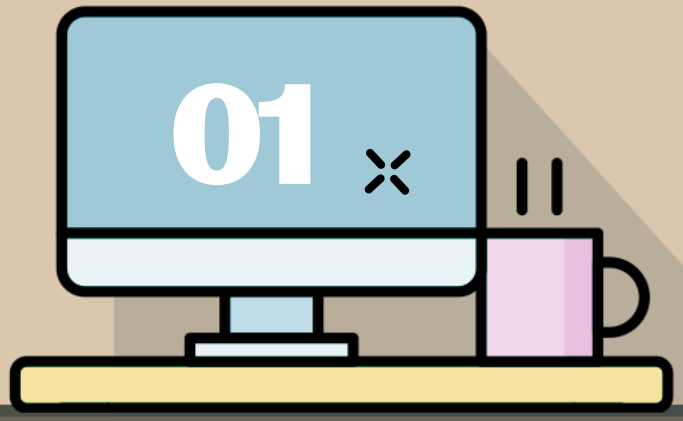
07강

함수와 기억 클래스(2)

컴퓨터과학과 이병래 교수

학습목차

1. 매개변수를 통한 자료 전달
2. 유효범위와 기억 클래스



매개변수를 통한 자료 전달

1. 함수의 실행

함수의 호출과 자료 전달

- 함수 호출: 처리에 필요한 자료를 피호출 함수의 매개변수에 전달함
- 복귀: 처리 결괏값을 함수 호출식의 값으로 반환함

```
int main()
{
    int d;
    d = sum(10, 20);
    printf("%d\n", d);
}
```

```
int sum(int x, int y)
{
    int z;
    z = x + y;
    return z;
}
```

30

2. 매개변수 사이의 자료 전달 방법

- 값에 의한 자료 전달(call by value)
  - C 언어의 기본적인 자료 전달 방식
  - 실 매개변수의 **값**을 형식 매개변수에 **복사**함
- 참조에 의한 자료 전달(call by reference)
  - 실 매개변수의 값이 들어 있는 곳의 **주소**를 전달함
  - C 언어에서는 직접 지원하지는 않지만, **포인터**를 이용하여 유사한 처리를 할 수 있음

3. 값에 의한 자료 전달

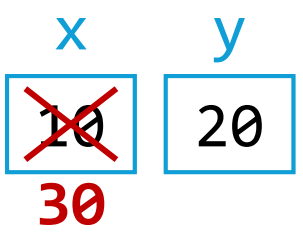
자료 전달 방법

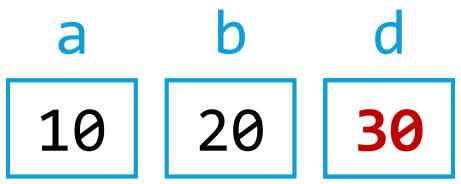
- 실 매개변수의 값을 형식 매개변수에 복사함
  - 실 매개변수: 원본
  - 형식 매개변수: 복사본
    - ▷ 자료 전달 후 원본과 복사본은 완전히 별개임
- 피호출 함수에서 형식 매개변수를 수정해도 실 매개변수는 원래의 값이 그대로 유지됨
  - 복사본만 수정한 것이므로 원본은 그대로 보존됨

3. 값에 의한 자료 전달

값에 의한 자료 전달의 예(1)

```c
#include <stdio.h>
int sum(int x, int y) {
    x += y;
    return x;
}
int main() {
    int a = 10, b = 20, d;
    d = sum(a, b);
    printf("%d + %d = %d\n", a, b, d);
}
```






```
10 + 20 = 30
```

3. 값에 의한 자료 전달

값에 의한 자료 전달의 예(2)

```
1  #include <stdio.h>
2  void swap(int x, int y);
3  int main() {
4      int a = 3, b = 5;
5      printf("호출 전 a = %d, b = %d\n", a, b);
6      swap(a, b);
7      printf("호출 후 a = %d, b = %d\n", a, b);
8  }
9  void swap(int x, int y) {
10     int temp = x;
11     x = y;
12     y = temp;
13     printf("함수 내 x = %d, y = %d\n", x, y);
14 }
```

a	b
3	5

x	y	temp
~~3~~ 5	~~5~~ 3	3

```
호출 전 a = 3, b = 5
함수 내 x = 5, y = 3
호출 후 a = 3, b = 5
```

4. 참조에 의한 자료 전달

자료 전달 방법

- 실 매개변수의 값이 들어 있는 곳의 주소를 형식 매개변수에 전달함
  - 주소를 전달하기 위해 형식 매개변수를 '포인터' 형으로 선언함
- 피호출 함수에서 형식 매개변수를 사용하는 것은 실 매개변수를 직접 사용하는 것과 동일함
  - 피호출 함수에서 실 매개변수의 값을 수정할 수 있어 매개변수를 통해 피호출 함수에서 호출 문장으로 자료를 전달할 수 있음

4. 참조에 의한 자료 전달

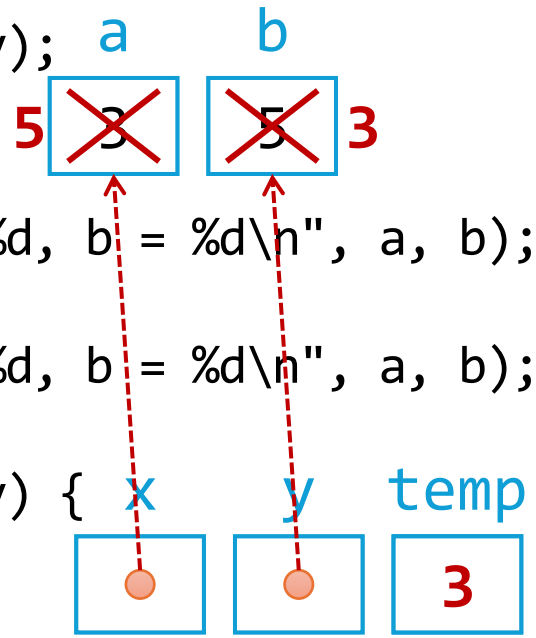
참조에 의한 자료 전달 방식의 사용 목적

- 피호출 함수에서 처리한 결과 자료를 매개변수를 통해 받아 오려는 경우
- 함수에 인수를 효율적으로 전달하려는 경우
  - 많은 자료 항목으로 구성되는 구조체와 같이 용량이 큰 인수를 전달하는 경우 자료를 형식 매개변수에 복사하는 대신 주소만 복사하면 효율적임

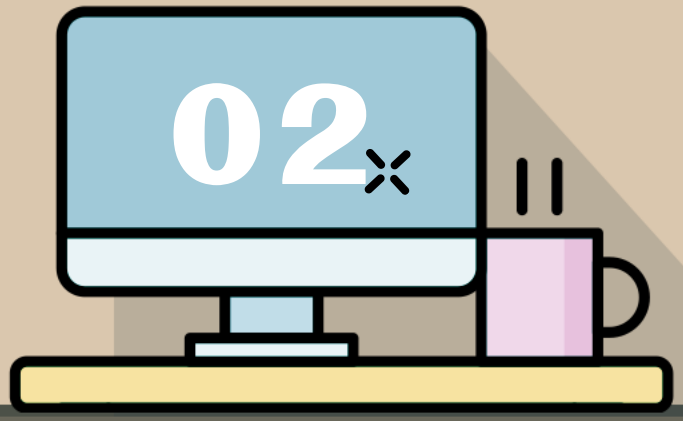
4. 참조에 의한 자료 전달

참조에 의한 자료 전달의 예

```c
#include <stdio.h>
void swap(int *x, int *y);
int main() {
    int a = 3, b = 5;
    printf("호출 전 a = %d, b = %d\n", a, b);
    swap(&a, &b);
    printf("호출 후 a = %d, b = %d\n", a, b);
}
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
    printf("함수 내 x = %d, y = %d\n", *x, *y);
}
```




```
호출 전 a = 3, b = 5
함수 내 x = 5, y = 3
호출 후 a = 5, b = 3
```



유효범위와 기억 클래스

1. 유효범위와 기억 클래스의 개념

- 유효범위(scope)란?
  - 변수나 함수 등의 명칭을 사용할 수 있는 영역
    - 지역변수, 전역변수
- 기억 클래스(storage class)란?
  - 변수의 존속기간(lifetime), 유효범위, 저장위치 등을 결정하는 개념
    - 자동변수, 정적변수, 외부변수, 레지스터 변수

2. 유효범위

- 지역변수(local variable)
  - 특정 범위 내에서만 통용되는 변수
    - 블록이나 함수 내부에 선언된 변수
      - ▷ 함수 헤더에 선언된 형식 매개변수 포함
    - 선언된 블록이나 함수 내에서만 사용 가능
- 전역변수(global variable)
  - 프로그램 전체에 걸쳐 사용될 수 있는 변수
  - 함수 외부에 선언된 변수
    - 동일 파일 내 또는 다른 파일에 선언될 수 있음
  - 명시적으로 초기화하지 않으면 0으로 초기화됨

2. 유효범위

```
#include <stdio.h>
int a;                  // 전역변수
void func(int b) {
    int c;              // 지역변수
    [          (가)          ]
    {
        double d;       // 지역변수
        [          (나)          ]
    }
}
double e;               // 전역변수
int main() {
    int a, f;           // 지역변수
    [          (다)          ]
}
```

a , b, c 사용 가능

a, b, c, d 사용 가능

전역변수 a의 이름을 가림(name hiding)

a, e, f 사용 가능

2. 유효범위

지역변수의 사용 예(1)

```
1  #include <stdio.h>
2  void fcn1();
3  int main() {
4      int i = 10;   // main() 함수의 지역변수
5      printf("main i = %d\n", i);
6      fcn1();
7      printf("main i = %d\n", i);
8  }
9  void fcn1() {
10     int i;    // fcn1() 함수의 지역변수
11     i = 20;   // main() 함수의 i와는 별개의 변수임
12     printf("fcn1 i = %d\n", i);
13 }
```

```
main i = 10
fcn1 i = 20
main i = 10
```

2. 유효범위

지역변수의 사용 예(2)

```
A: x = 2, y = 4
B: x = 5, y = 5
A: x = 2, y = 5
```

```c
#include <stdio.h>
int main() {
    // 블록 A
    int x = 2, y = 4;
    printf("A: x = %d, y = %d\n", x, y);
    {  // 블록 B
       int x;
       x = 5;
       y++;
       printf("B: x = %d, y = %d\n", x, y);
    }
    printf("A: x = %d, y = %d\n", x, y);
}
```

블록 A의 x와 y의 유효범위

블록 B의 x의 유효범위

2. 유효범위

전역변수의 사용 예

```
1  #include <stdio.h>
2  void fcn1();
3  int x;   // 전역변수(초기화 생략 → 0으로 초기화됨)
4  int main() {
5      printf("1) x = %d\n", x);
6      fcn1();
7      printf("2) x = %d\n", x);
8  }
9  void fcn1() {
10     x++;   // x는 전역변수이므로 어디서든 사용 가능함
11 }
```

```
1) x = 0
2) x = 1
```

2. 유효범위

지역변수와 전역변수의 비교

- 동일한 이름을 갖는 여러 변수의 유효범위에 속하는 경우 가장 안쪽 블록에 선언된 지역변수가 우선함

```
int a = 10;                    // 전역변수
int main() {
    int a = 20;                // 지역변수1
    {
        int a = 30;  // 지역변수2
        printf("%d\n", a);  --------> 30을 출력함
    }
    printf("%d\n", a);  ----------> 20을 출력함
}
```

- 전역변수는 프로그램 선두에 선언하는 것을 권장함

2. 유효범위

지역변수와 전역변수의 비교

- 가급적 지역변수를 사용하는 것이 좋음
  - 함수의 독립성 향상
  - 디버깅이 용이함
  - 기억공간 절약
  - 재귀호출(recursive call)을 하는 함수의 작성

3. 기억 클래스

변수의 기억 클래스 종류

- 자동 변수(auto)
- 정적 변수(static)
- 외부 변수(extern)
- 레지스터 변수(register)

변수를 선언할 때 기억 클래스를 지정함

▷ 예: `auto int a;`

기억 클래스에 따라 초기화, 존속기간, 유효범위의 차이가 있음

3. 기억 클래스

자동변수

- 지역변수에 적용될 수 있는 기억 클래스
- 예약어 'auto'를 지정하여 선언함
  - 디폴트가 auto이므로 생략할 수 있음
- 존속기간(lifetime): 소속 함수 또는 블록에 진입할 때 생성되고, 빠져나갈 때 제거됨
- 유효범위(scope): 소속 함수나 블록 내부
- 명시적으로 초기화하지 않으면 미지의 값을 갖게 됨

  ⇨ 적절한 초기화를 할 필요가 있음

3. 기억 클래스

자동변수의 예

```
1  #include <stdio.h>
2  int main() {
3      int i = 1;
4      auto int j = 2;
5      { /* 블록1 */
6          int i = 3;
7          { /* 블록2 */
8              int i = 4;
9              printf("블록2의 i = %d\n", i);
10             printf("블록2의 j = %d\n", j);
11         }
12         printf("블록1의 i = %d\n", i);
13     }
14     printf("main() 함수 내의 i = %d\n", i);
15 }
```

```
블록2의 i = 4
블록2의 j = 2
블록1의 i = 3
main() 함수 내의 i = 1
```

3. 기억 클래스

정적변수

- 예약어 ‘static’을 지정하여 선언함
- 존속기간(lifetime): 프로그램을 시작할 때 생성되어 프로그램을 마칠 때까지 유지됨
- 선언 위치별 유효범위(scope)
  - 함수나 블록에 선언: 소속 함수 또는 블록 내에서만 사용할 수 있음
  - 함수 외부에 선언: 소속 파일에서만 사용할 수 있음
- 초깃값을 지정하지 않으면 자동적으로 0으로 초기화됨

3. 기억 클래스

정적변수의 사용 예

```c
#include <stdio.h>
void test();
int main() {
    int i;
    for (i=0 ; i < 3 ; i++)
        test();
}
void test() {
    auto int a = 0;
    static int s = 0;
    printf("auto = %d, static = %d\n", a, s);
    ++a;
    ++s;
}
```

```
auto = 0, static = 0
auto = 0, static = 1
auto = 0, static = 2
```

3. 기억 클래스

외부변수

- 예약어 'extern'을 지정하여 선언함
- 함수 외부에서 정의된 변수라는 것을 의미함
- 존속기간(lifetime): 프로그램을 시작할 때 생성되어 프로그램을 마칠 때까지 유지됨
- 유효범위(scope): 프로그램 전체 범위
  - 다른 파일에 외부변수로 선언된 변수를 사용할 수 있음
- 초깃값을 지정하지 않으면 자동적으로 0으로 초기화됨

3. 기억 클래스

외부변수의 사용 예(1)

```
1  #include <stdio.h>
2  int i = 10;
3  int j = 20;
4  int main() {
5      extern int i;
6      extern int k;
7      int j = 100;
8      printf("i = %d, j = %d, k = %d\n", i, j, k);
9  }
10 int k = 50;
```

```
i = 10, j = 100, k = 50
```

3. 기억 클래스

외부변수의 사용 예(2)

5-13.c

```
1 #include <stdio.h>
2 void extern_ex();
3 char s[100];
4 int main() {
5   printf("문자열을 입력하세요: ");
6   scanf("%s", s);
7   printf("입력 문자열 %s는 전역변수 s에 저장\n", s);
8   extern_ex();
9 }
```

```
문자열을 입력하세요: KNOU
입력 문자열 KNOU는 전역변수 s에 저장

_
```

3. 기억 클래스

외부변수의 사용 예(2)

5-13_1.c

```c
#include <stdio.h>
extern char s[];
void extern_ex() {
  printf("외부변수 s의 값은 %s입니다.\n", s);
}
```

```
문자열을 입력하세요: KNOU
입력 문자열 KNOU는 전역변수 s에 저장
외부변수 s의 값은 KNOU입니다.
```

3. 기억 클래스

레지스터 변수

- 예약어 ‘register’를 지정하여 선언함
- CPU의 레지스터에 자료를 저장할 것을 요청함
  - 매우 자주 사용되는 변수에 지정하여 실행 속도를 높이려는 목적으로 사용함
  - 예: 반복문의 카운터 변수
- 존속기간 및 유효범위, 초깃값: 자동변수와 동일함

3. 기억 클래스

■ 레지스터 변수의 사용 예

```
1 #include <stdio.h>
2 int main() {
3     register int i;
4     int sum = 0;
5     for (i = 0 ; i <= 10 ; ++i)
6         sum += i
7     i -= 1;
8     printf("i = %d, sum = %d\n", i, sum);
9 }
```

```
i = 10, sum = 55
```

정리하기

- 함수 호출식에서 피호출 함수에 자료를 전달하기 위해 함수에 선언되는 매개변수를 사용한다.
- 값에 의한 자료 전달은 실 매개변수의 값을 형식 매개변수에 복사하는 방식으로, 함수 호출시의 실매개변수 값을 그대로 유지한다.
- 참조에 의한 자료 전달은 실 매개변수의 값이 들어 있는 곳의 주소를 형식 매개변수에 전달하는 방식으로, C에서는 포인터를 이용하여 구현한다.
- 지역변수는 변수를 선언한 블록이나 함수 내에서만 사용될 수 있으며, 전역변수는 프로그램 전체에 걸쳐 사용될 수 있다.

정리하기

- 자동변수는 소속 함수 또는 블록에 진입할 때 생성되고, 빠져나갈 때 제거되는 변수로, 지역변수의 기본적인 기억 클래스이다.
- 정적변수는 ‘static’을 지정하여 선언하며, 프로그램이 실행되는 동안 계속 유지된다.
- 외부변수는 함수 외부에 정의된 전역변수로, 다른 파일에 선언된 변수를 연결하기 위해 사용한다.
- 레지스터 변수는 매우 자주 사용되는 변수를 CPU 레지스터에 저장할 것을 요청하여 실행 속도를 높이려는 것이다.

08강

다음시간 안내

배열과 포인터(1)